

DSA Patterns and Data Structures for FAANG Interviews

Executive Summary: This report surveys the *key* data structures and algorithmic patterns crucial for FAANG-level coding interviews. We cover fundamental data structures (arrays, lists, trees, graphs, hash tables, etc.) and algorithmic patterns (two-pointers, sliding window, BFS/DFS, dynamic programming, greedy, backtracking, etc.). For each topic/pattern we give a concise definition, typical use cases, time/space complexity, representative example problems (with brief solution approach), common variations or tricks, and an estimated FAANG interview priority (High/Medium/Low). Authoritative sources (GFG, LeetCode, Tech Interview Handbook, etc.) are cited throughout ¹ ² ³. Finally, we provide consolidated tables of topics and a recommended study flow (mermaid chart), plus 10–15 curated practice problems with links.

Data Structure Topics

Arrays (High Priority) ¹ ⁴

Definition & Use: A dynamic or static sequence of elements indexed by position. Arrays allow **$O(1)$** random access by index, but insertion/deletion (except at the end) costs **$O(n)$** due to shifting ⁵. They are the most basic container for coding interviews and underlie many other problems.

When to Apply: Use an array or list to store/iterate ordered data, implement sliding windows, two-pointers, cyclic sorts, etc. Common in any problem involving sequences, e.g. sorting, scanning, partitioning.

Time/Space Complexity: Access $O(1)$, search $O(n)$ (or $O(\log n)$ if sorted with binary search), insert/delete $O(n)$ (worst case) ⁵.

Examples: Finding max/min, scanning or partitioning. E.g. **Maximum Subarray Sum** (Kadane's algorithm), **Cyclic Sort** (place numbers 1..n in place) ⁶, **Dutch National Flag** (3-way partition). Most patterns like two-pointers/sliding-window use arrays.

Variations/Tricks: Use two indices to traverse from ends or both forward (see "Two Pointers" below). Cycle-swap technique (put each element in its correct index) solves some "find missing/duplicate" problems ⁶. Beware slicing/copying cost $O(n)$ ⁷.

Interview Frequency: High – arrays appear in most interviews ¹.

Strings (High Priority) ⁸

Definition & Use: A sequence of characters (often treated as char array). Strings inherit array properties but come with string-specific operations. Used for problems on substrings, pattern matching, or any character processing.

When to Apply: Any problem on text or pattern search. Often solved with sliding-window (e.g. longest substring problems) or hashing (like anagram detection), or dynamic programming (edit distance).

Time/Space Complexity: Similar to arrays: index access $O(1)$, concatenation or substring (depending on language) may cost $O(n)$.

Examples: **Longest Palindromic Substring** (expand around center or DP), **Valid Anagram** (count chars with hash table), **Min Window Substring** (sliding window with frequency map).

Variations/Tricks: Use rolling hash or two-pointer / sliding window for subarrays of characters. Convert

to array of codes for bit tricks. Watch for UTF encoding issues.

Interview Frequency: High – strings are extremely common ⁸ .

Linked Lists (Medium Priority) ⁹

Definition & Use: A sequence of nodes where each node points to the next (singly or doubly linked). Insertion/deletion at a node is $O(1)$ if you have a pointer to it, but accessing index i is $O(n)$. Useful for dynamic memory ordering.

When to Apply: Problems involving pointer manipulation, reversing links, or where insertion/deletion in the middle is required. Also used when you want to use $O(1)$ extra space to reverse or detect structure.

Time/Space Complexity: Access/search $O(n)$, insertion or deletion (given node) $O(1)$. Traversal $O(n)$. Extra pointers require more memory per element.

Examples: **Reverse Linked List** (in-place pointer reversal in one pass), **Detect Cycle** (Floyd's Tortoise and Hare, see "Fast/Slow Pointers" below), **Merge Two Sorted Lists**.

Variations/Tricks: Two-pointer patterns (see below) frequently apply to linked lists (find middle, remove n th from end). Dummy head nodes can simplify edge cases. Flatten nested lists via recursion.

Interview Frequency: Medium – core operations and reversal are common; cyclical detection/populating pointers come up often ¹⁰ .

Stacks & Queues (Medium Priority) ¹¹

Definition & Use: Stack (LIFO) and Queue (FIFO) are fundamental ADTs. Both support $O(1)$ insert/delete operations at one end (stack) or both ends (deque). They implement backtracking or breadth-first search, respectively.

When to Apply: Use a stack for DFS, expression parsing (infix $\rightarrow$ postfix), or monotonic stack patterns. Use a queue for BFS, sliding window optimisations, or as auxiliary in some DP.

Time/Space Complexity: All stack/queue ops (push/pop, enqueue/dequeue) are $O(1)$.

Examples: **Valid Parentheses** (use stack to match brackets), **MinStack** (stack with tracking min).

Sliding Window often uses a deque to store indices (e.g. Monotonic Queue for max/min in window).

Variations/Tricks: Monotonic stacks/queues (maintaining sorted order) solve "next greater element" or sliding-window maxima in linear time. Using two stacks can implement a queue (and vice versa) if needed.

Interview Frequency: Medium – common concept checks; monotonic stacks are *emerging as important patterns* ¹² ¹³ .

Hash Tables (Medium Priority) ¹⁴ ¹⁵

Definition & Use: A key-value map/dictionary that offers average $O(1)$ insert, lookup, and delete. A powerful tool to replace nested loops by lookup tables.

When to Apply: Anytime you need constant-time membership, counts, or lookups. Pattern often: transform an $O(n^2)$ brute-force solution into $O(n)$ by storing previously seen elements. Used in two-sum, anagrams, frequency counting, sliding-window with counts, etc.

Time/Space Complexity: Average $O(n)$ time for building or searching n items; worst-case can degrade but usually disregarded. Space $O(n)$ for storing keys.

Examples: **Two Sum** (store complement indices), **Group Anagrams** (map sorted string to bucket) ¹⁵ , **Longest Consecutive Sequence** (hash-set scan) ¹⁵ , **Top K Frequent Elements** (map + heap).

Variations/Tricks: Hashing tricks (like rolling hash for substring matching, or bitmask as key). Use hash to implement memoization (cache results). Beware of collision attacks (not relevant in interviews).

Interview Frequency: High – extremely common; often used even in problems not obviously about hashing ¹⁵ .

Trees (High Priority) ¹⁶

Definition & Use: Hierarchical structures with nodes having children. Common types: **Binary Trees** (each node up to 2 children), **Binary Search Trees** (BSTs enforce left<node<right ordering), and **Heaps** (priority tree).

When to Apply: Any structured data questions (search, insert, traverse). Trees model file systems, parsed expressions, game states, etc.

Time/Space Complexity: Basic traversals (pre/in/post-order) are $O(n)$ time and space (for recursion) on an n -node tree. BST search/insert/delete average $O(\log n)$ (balanced) or worst $O(n)$ (skewed).

Examples: **Binary Tree Level-Order Traversal (BFS)**, **Invert/Serialize Tree (DFS)**, **Lowest Common Ancestor**, **BST Operations**, **Heap Insert/DeleteMin**, **Trie** (prefix tree, see below).

Variations/Tricks: Many tree problems use recursion or stack. Convert tree problems to arrays (index heap, etc.). For binary search trees, use inorder/traversal techniques. Augmented trees (segment tree, Fenwick) appear rarely but can be mentioned if time permits.

Interview Frequency: High – binary trees/BST questions are frequent, requiring mastery of traversal techniques ¹⁶.

Tries (Prefix Trees) (Medium Priority) ¹⁷

Definition & Use: A **trie** is a tree structure for storing strings by characters. Each path from the root represents a prefix. Lookups run in $O(m)$ time (m = key length), independent of number of entries ¹⁷.

When to Apply: When you have many string keys and need prefix queries (autocomplete), spell-check, or dictionary search. Also useful for problems with word grids (Word Search II).

Time/Space Complexity: Insertion/search $O(m)$ per word. Space $O(n \cdot m)$ in worst case (n words of length m , shared prefixes reduce this).

Examples: **Implement Trie (Add, Search, StartsWith)** (LeetCode 208) ¹⁸, **Word Search II** (backtracking + trie) ¹⁹, **Replace Words** (prefix matching).

Variations/Tricks: Use an array or hash for children links. Compress with a **radix tree** for contiguous nodes (advanced). A trie built on binary representations (bitwise trie) can answer XOR maximization queries.

Interview Frequency: Medium – although less frequent than arrays, tries appear in prefix/search problems ¹⁷.

Hash-Indexed Structures (Medium Priority)

This includes **Hash Table** (discussed above) and **Hash Sets**. Often used interchangeably in solutions: e.g. keep a hash-set for visited nodes (graph traversals) or to detect duplicates.

Examples: Deduplication, cycle detection in graph (visited set), hashing for rolling hash (Rabin-Karp algorithm), etc.

Interview Frequency: Medium.

Graphs (High Priority) ²⁰ ²¹

Definition & Use: A set of vertices connected by edges (directed/undirected). Graphs represent networks, grids (as implicit graph), dependencies, etc.

When to Apply: Connectivity, path-finding, or ordering problems. If a problem involves “network” or “map” or “matrix traversal”, consider graph algorithms.

Time/Space Complexity: Representations: adjacency list is most common (space $O(V+E)$). **BFS/DFS** traversals take $O(V+E)$ ²¹. Shortest paths: Dijkstra's $O((V+E)\log V)$; Topological sort $O(V+E)$.

Examples: **Number of Islands** (grid BFS/DFS), **Course Schedule** (topological sort), **Word Ladder** (BFS on word graph), **Graph Valid Tree** (union-find or BFS cycle-check) ²².

Variations/Tricks: Use BFS for shortest path in unweighted graphs. Use DFS for cycle detection (track recursion stack) or connectivity. Use in-degree counting or DFS for topo sort. Edge cases: disconnected components.

Interview Frequency: High – graphs and trees are grouped together as advanced DS, and graph questions (BFS/DFS) appear commonly ²³ ²⁴ .

Heaps / Priority Queues (Medium Priority)

Definition & Use: A heap is a binary tree structure (or array-based) where parent $\leq$ children (min-heap) or $\geq$ (max-heap). A **priority queue** supports extract-min (or max) in $O(\log n)$. Useful for selection problems.

When to Apply: Whenever you need to repeatedly access/remove the largest/smallest element. Common in: Top K problems, merging sorted lists, scheduling.

Time/Space Complexity: Insertion/extraction $O(\log n)$, peek $O(1)$. Build-heap (n elements) $O(n)$.

Examples: **Top K Frequent Elements** (min-heap of size k) ¹⁵ , **K-th Largest Element in Array**, **Merge k Sorted Lists** (k-way merge via heap) ²⁵ , **Median of Data Stream** (two-heap method) ²⁶ .

Variations/Tricks: Use two heaps (max-heap and min-heap) to maintain medians ²⁶ . Use a min-heap of size K to track top K; or a max-heap to sort partially. Be careful with Python's heapq (min-heap only) – use negative values or use `nlargest` .

Interview Frequency: Medium – heaps come up in selection/ranking problems ²⁶ ²⁵ .

Interval / Segment Problems (Medium Priority)

Definition & Use: Many problems involve intervals (ranges) or segments. Patterns: **Merge Intervals**, **Interval Scheduling**, **Line Sweep**.

When to Apply: Given a set of intervals (start, end), sort by start or end and process them. Useful for scheduling (meeting rooms), merging overlapping bookings, or computing area under intervals.

Time/Space Complexity: Sorting $O(n \log n)$ often dominates; then one pass $O(n)$.

Examples: **Merge Overlapping Intervals** (sort by start, merge if overlap), **Meeting Rooms II** (min-heap tracking end times, $O(n \log n)$), **Insert Interval**, **Interval List Intersections**.

Variations/Tricks: Use min-heap to track current intervals (e.g. next earliest finish). For union of intervals, scan sorted endpoints. Related: calendar booking problems, or the “skyline” problem.

Interview Frequency: Medium – interval problems are common (especially merging/scheduling) but not the most frequent ²⁷ .

Matrix / Grid (High Priority)

Definition & Use: A 2D array treated as a graph of cells.

When to Apply: Common in pathfinding or flood-fill problems. Usually solved via BFS/DFS on a grid.

Time/Space Complexity: BFS/DFS in an $m \times n$ grid is $O(mn)$ time and space.

Examples: **Number of Islands** (count connected components of 1's), **Surrounded Regions**, **Pacific Atlantic Water Flow**.

Variations/Tricks: Four-direction or 8-direction movement. Use visited matrix to avoid repeats. Often combine with DP (DP on grids) or BFS (shortest path in maze).

Interview Frequency: High – matrix/grid DFS/BFS is very common.

Advanced (Lower Priority)

- **Dynamic Programming (DP)** – treated below as an algorithmic pattern (often frequent, see later). TIH lists it as Low priority ²⁸ , but many interviews do ask DP/Greedy (especially dynamic programming).

- **Binary Representations & Bitsets** – e.g. bitsets, bitmask DP (exhaustive subsets for $n \leq 20$).
- **Math/Number Theory** – e.g. gcd, primes (Sieve), geometry – rarely core unless specifically asked. TIH tags these Low ²⁹.
- **Geometry** – e.g. computational geometry (line sweep), not common in FAANG coding.
- **Divide-and-Conquer / Recursion** – general strategy; see algorithm patterns below.
- **Graph Advanced Algorithms** – Dijkstra, A*, max-flow, etc. Rarely needed in basic interviews (TIH: “almost never” ³⁰).

Algorithmic Patterns

Two Pointers Technique (High Priority) ³¹

Definition: Use two indices traversing the array (or list), either one from start and one from end or both from start, to solve problems more efficiently ³¹. This often transforms an $O(n^2)$ brute-force into $O(n)$.

When to Apply:

- **Sorted input:** find two-sum, closest pair, etc. (Example: *Two Sum II* – one pointer from each end ³²).
- **Pair/range problems:** search for pairs/triplets satisfying a condition in an array (e.g. *3Sum*, *Container with Most Water*).
- **Linked lists (fast/slow):** detect cycles or find middle (see next).
Complexity: Typically $O(n)$ time, $O(1)$ space (in-place) for simple two-pointer; sliding window is a variant (below) usually $O(n)$.

Examples:

- *Two Sum II* (sorted array): one pointer at left, one at right, move inward based on sum ³².
- *Container With Most Water*: maximize area with left/right pointers.
- *3Sum*: sort then fix one number and use two pointers for the remaining two.

Variations/Tricks:

- **Slow/Fast pointers:** one pointer advances twice as fast to detect loops or find list middle (see below).
- **Cycle Sort / Cyclic Sort:** similar idea of moving elements to “correct” index with swaps (works for numbers $1..n$) ⁶.
Interview Frequency: High – appears in many array problems (often overlapped with sliding window) ³¹.

Sliding Window (High Priority) ³

Definition: Maintain a “window” (subarray) of the array/string and slide it (expanding/shrinking) to satisfy a condition. Avoids re-scanning the whole array for each position ³.

When to Apply:

- Problems on subarrays/substrings, e.g. find max sum of size K, longest substring meeting some constraint (unique chars, at most k distinct, etc) ³.
- When you need to handle dynamic ranges or fixed-size windows.
Complexity: $O(n)$ time (each element enters/exits window once) and typically $O(1)$ extra space

(ignoring output).

Examples:

- *Maximum Sum Subarray of Size K* (fixed window): use running sum and subtract outgoing element 3.

- *Longest Substring Without Repeating Characters*: expand window until a repeat, then shrink from left.

- *Minimum Window Substring*: two-pointer shrink/expand window with hash counts.

Variations/Tricks:

- **Fixed-size window**: incrementally update sum/count.
- **Variable-size**: use two pointers (same as sliding-window two-pointer) and maintain counts in a hash/map.
- Often combined with frequency map (hash table) to check constraints on characters.

Interview Frequency: High – a core pattern for substring/subarray problems 3.

Fast and Slow Pointers (Hare & Tortoise) (Medium Priority)

Definition: Two pointers move at different speeds (fast moves 2 steps, slow 1 step).

When to Apply: Detect cycles and find cycle entry/middle in linked lists or arrays (Floyd's algorithm).

Also used in detecting repeat patterns.

Complexity: $O(n)$ time, $O(1)$ space.

Examples:

- *Linked List Cycle Detection (Floyd's)*: if fast ever equals slow, there is a cycle.
- *Find Cycle Start*: after detection, reset one pointer to head and move both at speed 1; the meeting point is cycle start.
- *Middle of Linked List*: just stop when fast reaches end.

Variations/Tricks: Can extend to "remainder" states in number sequences (repeat detection). Always check edge cases (empty list, single node).

Interview Frequency: Medium – often part of linked-list problems and sometimes implicit cycle detection.

Breadth-First Search (BFS) and Depth-First Search (DFS) (High Priority) 21

Definition: Standard graph/tree traversal algorithms. **BFS** explores neighbors level by level (using a queue), **DFS** explores as deep as possible (using recursion or explicit stack). Both run in $O(V+E)$ time 21.

When to Apply:

- **BFS**: shortest-path in unweighted graphs, connectivity in layers, level-order tree traversal.
- **DFS**: explore all paths, detect cycles, backtracking, topological sort (via DFS finishing times).
Complexity: $O(V+E)$ time for graphs; $O(n)$ for n -node tree. Space $O(V)$ worst-case (visited set/stack).

Examples:

- *Tree Level Order Traversal*: use queue (BFS).
- *Binary Tree Traversal (In/Pre/Post)*: use DFS recursively or stack.

- *Number of Islands*: BFS/DFS to mark connected land.
- *Topological Sort*: Kahn's algorithm (BFS on zero in-degree) or DFS post-order ³³.
- *Graph Connected Components / Bipartiteness*: BFS/DFS grouping.

Variations/Tricks:

- Always mark visited to avoid infinite loops.
- For grid problems, treat 2D matrix as a graph (up/down/left/right neighbors).
- Use BFS level-order to count distance or steps.

Interview Frequency: High – graph/tree traversals are ubiquitous ²³.

Binary Search (High Priority)

Definition: Divide-and-conquer technique on sorted data: compare target to middle element to halve search space.

When to Apply: Anytime a sorted structure is given or monotonic condition holds. Use on sorted arrays, binary-search trees (implicit), or even on function domains (binary search on answer).

Complexity: $O(\log n)$ time, $O(1)$ space (iterative).

Examples:

- *Standard*: search in sorted array or range queries (e.g. "find first $\geq X$ ").
- *Variants*: Search in rotated sorted array, find peak element, search in a matrix with sorted rows/columns.
- *Binary Search on Answer*: e.g. allocate resources, split tasks (e.g. capacity to ship packages in D days).

Variations/Tricks:

- Watch for integer overflow when computing $\text{mid} = (\text{lo} + \text{hi}) // 2$; use $\text{lo} + (\text{hi} - \text{lo}) // 2$.
 - Lower/upper bound (first true) binary search.
 - Use binary search on a function (predicate) if monotonic.
- Interview Frequency: High** – one of the most fundamental patterns for sorted data.

Divide and Conquer (Medium Priority)

Definition: Recursively split problem into subproblems, solve each, then combine.

When to Apply: Sorting (merge sort, quicksort), tree construction, selection (find kth smallest by quickselect).

Complexity: Often $O(n \log n)$ or better. E.g. merge sort $O(n \log n)$, quickselect average $O(n)$.

Examples:

- *Merge Sort*: split array, sort halves, merge.
- *Quickselect*: select pivot, partition, recurse only into relevant side.
- *Closest Pair of Points*: divide plane recursively (computational geometry).

Variations/Tricks:

- Combine with other patterns: e.g. in a sorted array use binary search.
- Optimize by truncating recursion when possible.

- Tail recursion elimination or convert to loops when needed.

Interview Frequency: Medium – not asked explicitly often, but implicit in many algorithms (tree recursion, array sorting, selection).

Dynamic Programming (DP) (Medium to High Priority) ³⁴

Definition: Technique to solve problems with overlapping subproblems and optimal substructure by storing (memoizing) intermediate results ³⁴. Avoids exponential recomputation.

When to Apply: Optimization/counting problems on sequences, trees, or grids. Typical keywords: “maximum/minimum,” “number of ways,” “optimal,” etc ²⁴.

Complexity: Depends on state space; e.g. for an array DP with n states and $O(1)$ transitions, $O(n)$; for two-string LCS, $O(nm)$. Usually polynomial time, linear or quadratic. Space often $O(n)$ or $O(n^2)$ for table.

Examples:

- *Climbing Stairs* (simple 1D DP) ³⁵.
- *Longest Common Subsequence* (2D DP) ³⁵.
- *0/1 Knapsack, Coin Change, Edit Distance, Word Break* (classic problems) ³⁵.
- *DP on Trees*: e.g. tree DP for tree shape problems (harder).

Variations/Tricks:

- **Memoization vs Tabulation:** top-down recursion vs bottom-up loops.

- **State Representation:** carefully choose minimal state (indices, masks).
- **Optimization:** space optimized (keep only last row for 1D).
- Some problems use bitmask DP (subset DP) or “DP + bit operations.”

Interview Frequency: Medium – DP is tricky but appears often; sometimes interviews emphasize iterative solutions (TIH flagged DP as Low priority ²⁸, but many problems are DP disguised).

Greedy (Medium Priority) ³⁶

Definition: At each step, choose the best immediate (local) option, hoping it leads to a global optimum. Does **not** revisit decisions ³⁶. Only valid when local optimum = global optimum.

When to Apply: Optimization problems (minimize cost, maximize coverage) where sorting/selection yields an obvious choice. Common when tasks/jobs with deadlines, intervals, or resources.

Complexity: Usually $O(n \log n)$ due to initial sort, then $O(n)$ pass. If no sort needed, $O(n)$.

Examples:

- *Jump Game* (greedy jump farthest) ³⁷.
- *Gas Station* (greedy sum check) ³⁷.
- *Interval Scheduling / Minimum Platforms*: sort intervals by end/start, pick accordingly.
- *Dijkstra's Algorithm*: greedy for shortest path (using a heap).

- *Activity Selection*: pick earliest finishing job.

Variations/Tricks:

- Identify a greedy-choice property (often by proof or counterexample).

- Sort by a key that simplifies decision (e.g. sort gas stations by distance).
- Combine with heap for optimization (e.g. scheduling with resources).

Interview Frequency: Medium – many classic problems (interval, scheduling) use greedy ³⁷.

Backtracking (Medium Priority) ³⁸

Definition: Systematically explore all candidates for the solution, abandoning ("backtracking") partial solutions as soon as they're invalid ³⁸ . Prunes search space exponentially.

When to Apply: Combinatorial generation or constraint problems: generate permutations/combinations, solve puzzles, or any "try all configurations but prune early."

Complexity: Worst-case exponential ($O(k^n)$ for branching factor k and depth n). But pruning often reduces it.

Examples:

- *Permutations / Combinations* generation ³⁹ .
- *N-Queens, Sudoku Solver* (constraint satisfaction) ³⁹ .
- *Word Search / Crossword* (DFS on grid) ⁴⁰ .
- *Subsets (Power set generation)* ⁴¹ .

Variations/Tricks:

- Always apply early pruning (e.g. in N-Queens, drop branch when conflict).
- Use recursion or iterative with stack.
- Often combined with other structures (e.g. build words with trie).

Interview Frequency: Medium – known as "hard" problems, but common in backtracking sets ³⁹ .

Bit Manipulation (Medium Priority) ⁴²

Definition: Use bitwise operations (AND, OR, XOR, NOT, shifts) on integers to solve problems by exploiting binary representation ⁴² .

When to Apply: Uniqueness/parity problems, operations on sets/flags, power-of-two checks, subset enumeration via bitmasks. If a problem hint involves even/odd or toggling bits, think bit ops.

Complexity: Usually $O(n)$ where n is number of elements (plus bit ops $O(1)$). Bitwise loops (count bits) cost $O(\log M)$ per number if needed.

Examples:

- *Single Number* (find non-duplicate using XOR) ⁴³ .
- *Hamming Weight* (count 1 bits in integer) ⁴³ .
- *Missing Number* (XOR of $0..n$).
- *Subset Generation*: use integers $0..(1<n)-1$ to represent subsets.

• *Power of Two check*: `(n & (n-1)) == 0` .

Variations/Tricks:

- XOR has nice properties: $a \oplus a = 0$, $0 \oplus a = a$.
- Use bitmask to compress state (e.g. DP for TSP).
- Shift operators for multiply/divide by 2.

Interview Frequency: Low-to-Medium – appears in coding tests but less frequent, often for "easy" problems ⁴² .

Prefix Sum (Medium Priority) ⁴⁴

Definition: Precompute the cumulative sums (or products) of an array so range queries can be answered in $O(1)$ ⁴⁴. More generally, accumulate any reversible operation.

When to Apply: Frequent range-sum queries or any problem with repeated subarray sums. Also for count differences (transform problems into prefix arrays).

Complexity: Preprocessing $O(n)$, then each query $O(1)$ (range $[L,R]$ via $\text{prefix}[R] - \text{prefix}[L-1]$). Memory $O(n)$ for prefix array.

Examples:

- *Range Sum Query* (build prefix to answer sum queries) ⁴⁴.
- *Subarray Sum Equals K* (store prefix-sums in hash map, then find how many $(\text{prefix}[i] - K)$ occurred) ⁴⁵ ⁴⁶.
- *Product of Array Except Self* (prefix/suffix products).

Variations/Tricks:

- 2D prefix sums (summed-area table) for matrix sums.
 - Prefix XOR or prefix bitmask for parity queries (count even/odd segments).
- Interview Frequency: Medium** – common trick for range queries, and appears in sliding-window contexts ⁴⁴.

Monotonic Stack (Medium Priority) ⁴⁷

Definition: A stack that maintains its elements in sorted (monotonic) order (increasing or decreasing) ⁴⁷. On each step, pop from the stack until the order property holds. Used to answer “next greater/smaller element” queries in $O(n)$.

When to Apply: Problems that require finding the next/previous greater or smaller element, or “histogram” problems. Any scenario with a fixed order scan but needing to “look ahead.”

Complexity: $O(n)$ time (each element pushed/popped at most once), $O(n)$ space for stack.

Examples:

- *Next Greater Element*: stack stores indices, pop until current is smaller ⁴⁷.
- *Daily Temperatures* (each day's next hotter day) ⁴⁷.
- *Largest Rectangle in Histogram* (classic monotonic stack) ⁴⁸.
- *Trapping Rain Water* (can be solved with stack or two pointers).

Variations/Tricks:

- Use either an increasing or decreasing stack depending on query (next greater vs next smaller) ⁴⁷.
- While scanning, when you pop an element, you know its “next greater” is the current element.

Interview Frequency: Medium – less common but appears in many “hard” array problems ⁴⁷.

Hash Map / Frequency Count (Medium Priority) ¹⁵

Definition: As above, using a hash (often Python dict or Java HashMap) to store frequencies, indices, or complements.

When to Apply: Finding pairs/triples (Two Sum), grouping by anagram, sliding-window with count of chars, counting occurrences. Any need for quick lookup of past info.

Complexity: $O(n)$ time using the hash for each lookup (amortized), $O(n)$ space.

Examples:

- *Two Sum*: store seen values in map to check complement in $O(1)$ ¹⁵.
- *Group Anagrams*: use sorted-string or char-count as key in map ¹⁵.
- *Longest Consecutive Sequence*: put in hash-set and expand neighbors ¹⁵.
- *Sliding Window Character Count*: map to enforce “at most k distinct” etc.

Variations/Tricks: Use composite keys (tuples) for multi-dimensional counts, or encode state as string. Detect patterns with K occurrences by map.

Interview Frequency: High (overlaps with Hash Tables) – one of the simplest tools that optimizes many naive solutions ¹⁵.

Union-Find (Disjoint Set) (Low-to-Medium Priority) ² ⁴⁹

Definition: Data structure for tracking a set of disjoint groups. Supports *union*(x,y) to merge groups and *find*(x) to get the group representative ⁴⁹. With path compression/rank, operations are near constant-time.

When to Apply: Connectivity problems: “are these two items connected?”, count connected components, cycle detection in undirected graph, Kruskal’s MST.

Complexity: Almost $O(1)$ (amortized $\alpha(n)$, inverse Ackermann) per operation; building union-find is $O(n)$ initially.

Examples:

- *Number of Connected Components in Graph* (Union all edges, then count reps) ².
- *Graph Valid Tree*: check if adding any edge forms a cycle (if union-find finds same parent) ⁵⁰.
- *Redundant Connection*: find extra edge that creates a cycle ⁵⁰.
- *Account Merge*: group accounts by emails with union-find ⁵⁰.

Variations/Tricks: Path compression and union by rank are key optimizations. Can also label components on the fly.

Interview Frequency: Medium – less common than BFS/DFS for connectivity but appears (often GFG/LintCode staple) ².

Topological Sort (Medium Priority) ⁵¹

Definition: Linear ordering of nodes in a DAG such that for every directed edge $u \rightarrow v$, u comes before v . Computes a valid sequence of tasks with dependencies.

When to Apply: Scheduling with prerequisites (courses/tasks), build order, ordering constraints.

Complexity: $O(V+E)$ – use BFS (Kahn’s algorithm) or DFS.

Examples:

- *Course Schedule (LeetCode 207)*: determine if all courses can be finished (detect cycle or produce order).
- *Alien Dictionary*: derive char order from word list.
- *Build Order*: standard interview example.

Variations/Tricks: In-degree counting with queue (Kahn), or DFS post-order stack and reverse. Check for cycles by seeing if ordering includes all nodes.

Interview Frequency: Medium – common in scheduling/dependency problems (TIH notes it in Graph section) ²³ ⁵¹.

Additional Patterns

- **Matrix Traversal:** BFS/DFS on grids (see Matrix/Grid above).

- **Geometric Sweep Line:** Rare (e.g. rectangle union, skyline) – beyond scope unless specified.
- **Bitmask DP (Subset DP):** Using bits to represent subset of $\{1..n\}$; common if $n \leq 20$.

Topic Summary Tables

Data Structures and Key Subtopics:

Data Structure	Key Concepts/Subtopics	Frequency
Arrays & Strings	Indexing, Two pointers, Sliding-window, Sort/Search (binary search)	High ¹
Linked Lists	Reversal (in-place), Cycle detection (fast/slow), Merge, Insert/Delete	Medium ⁹
Stacks/Queues/Deque	LIFO/FIFO operations, Monotonic Stack (next greater), Graph BFS (queue), Call stack (DFS)	Medium ¹¹
Hash Table / Hash Set	Key-value map (freq count, index map), Hash-set (visited)	High ¹⁴
Heaps/PriorityQueue	Min-/Max-heap operations, Top-K, Median-of-stream (two heaps)	Medium ⁵²
Trees (Binary/Trie)	Traversals (pre/in/post), BST operations, Depth/Height, LCA, Trie (prefix tree)	High ¹⁶ ¹⁷
Graphs	Adjacency list, BFS, DFS, Directed vs Undirected, Weighted vs Unweighted (Dijkstra), Topo sort, UF	High ²⁰ ²
Interval/Range	Merge intervals, Scheduling, 2D prefix-sum (matrix sums)	Medium
Grid/Matrix	2D traversal (DFS/BFS), DP in matrix (unique paths, island count)	High
Others	Bitwise ops, Mathematical (primes, gcd), Geometry, Recursion fundamentals	Low

Algorithmic Patterns:

Pattern	When to Use & Examples	Complexity	Freq.
Two Pointers	Two indices on sorted/unsorted data (e.g. <i>Two Sum II</i> , <i>3Sum</i> , <i>Container with Water</i>).	$O(n)$ time, $O(1)$ space ³¹	High
Sliding Window	Maintain subarray range (e.g. longest/shortest substring/sum problems) ³ .	$O(n)$ time ³	High
Fast/Slow Pointers	1 fast, 1 slow index (cycle detection, middle of list).	$O(n)$ time, $O(1)$ space	Medium
BFS / Level-Order	Unweighted shortest path or level-wise traversal (tree or graph).	$O(V+E)$ time ²¹	High

Pattern	When to Use & Examples	Complexity	Freq.
DFS / Pre/Post-Order	Explore paths/cycles; backtracking; topo sort (via post-order).	$O(V+E)$ time ²¹	High
Binary Search	Search in sorted data or on monotonic condition (rotated array, matrix search).	$O(\log n)$ time	High
Divide & Conquer	Split problem (merge sort, quickselect).	$O(n \log n)$ typical	Medium
Dynamic Programming	Overlapping subproblems + optimal substructure (e.g. <i>Climbing Stairs</i> , <i>LCS</i> , knapsack) ³⁴ .	Polynomial (often $O(n^2)$ or $O(nm)$)	Medium
Greedy	Locally optimal steps (when they lead to global optimum, e.g. <i>Jump Game</i> , <i>Gas Station</i> ³⁷ , scheduling by earliest finish).	Usually $O(n \log n)$ (sort)	Medium
Backtracking	Recursive search with pruning (all permutations, N-Queens) ³⁹ .	Exponential worst case	Medium
Bit Manipulation	Work on bits (XOR tricks, parity, power-of-two checks).	Usually $O(n)$ (plus bit ops) ⁴²	Low
Prefix Sum	Precompute cumulative sums/products (range queries or <i>Subarray Sum = K</i>) ⁴⁴ ⁴⁶ .	Preprocess $O(n)$, queries $O(1)$	Medium
Hash/Frequency Count	Store frequencies/indices for $O(1)$ lookup (e.g. <i>Two Sum</i> , <i>Anagrams</i>).	$O(n)$ time, $O(n)$ space ¹⁵	High
Union-Find	Connectivity/cycle in undirected graphs (friend groups, redundant connection) ² .	Nearly $O(1)$ per op ⁴⁹	Medium
Topological Sort	Order tasks with dependencies (course schedule) ⁵¹ .	$O(V+E)$ time ²¹	Medium
Monotonic Stack	Next greater/smaller element or histogram problems ⁴⁷ .	$O(n)$ time	Medium

Study Plan (Priority Flow)

```

flowchart TB
    A([Foundation: Arrays, Strings]) --> B([Core Structures: LinkedList, Stack, Queue, Hash])
    B --> C([Sorting & Searching: Binary Search, Sort, Recursion])
    C --> D([Trees & Graphs: Traversals, BSTs, Graph BFS/DFS])
    D --> E([Heaps/Tries & Intervals: PriorityQueues, Prefix Trees, Interval Scheduling])
    E --> F([Advanced Patterns: DP, Greedy, Backtracking, Bit Manipulation])
    style A fill:#ccf,stroke:#333,stroke-width:2px
    style F fill:#fcc,stroke:#333,stroke-width:2px
    subgraph High Priority Topics
        A
    end

```

```

        D
        E
    end
    subgraph Medium Priority Topics
        B
        C
    end
    subgraph Low Priority / Stretch
        F
    end
end

```

(Flowchart: start with fundamental arrays/strings; then core DS (lists, stacks, hash); then sorting/searching; then tree/graph; finally complex patterns.)

Practice Problems (Curated Examples)

Below are **15 representative problems** covering the above patterns and data structures (with sources):

- **Two Sum II (Sorted Array)** – LeetCode 167. Two-pointer approach (one left, one right) ³² .
- **Subarray Sum Equals K** – LeetCode 560. Prefix-sum + hash map to count range sums ⁴⁴ ⁴⁶ .
- **Longest Substring Without Repeating Characters** – LeetCode 3. Sliding-window with hash to maintain unique chars.
- **Daily Temperatures** – LeetCode 739. Monotonic stack for next greater element ⁴⁸ .
- **Trapping Rain Water** – LeetCode 42. Two-pointer or stack solution for container-type problem.
- **Container With Most Water** – LeetCode 11. Two-pointer greedy (maximize area).
- **Permutations** – LeetCode 46. Backtracking to generate all orderings ³⁹ .
- **N-Queens** – LeetCode 51. Backtracking with pruning for queen placements ³⁹ .
- **Climbing Stairs** – LeetCode 70. Simple DP (Fibonacci).
- **Longest Common Subsequence** – LeetCode 1143. 2D DP for sequence comparison ³⁵ .
- **Jump Game** – LeetCode 55. Greedy (track furthest reachable).
- **Meeting Rooms II** – LeetCode 253. Sort intervals and use min-heap of end-times.
- **Number of Islands** – LeetCode 200. Grid BFS/DFS to count connected components.
- **Course Schedule** – LeetCode 207. Graph topo-sort or cycle detection (DFS/union-find).
- **Top K Frequent Elements** – LeetCode 347. Hash-map to count + min-heap of size K.
- **Implement Trie (Prefix Tree)** – LeetCode 208. Build/search prefix structure ¹⁸ .

Each problem link leads to detailed statements; the strategies listed above (stack, DP, etc.) apply. References on patterns are cited so you can connect each pattern to practice problems ⁵³ ⁵⁴ .

Sources

We cite authoritative sources for definitions, complexities, and patterns:

- Tech Interview Handbook cheat sheets ¹ ²¹
- GeeksforGeeks tutorials (Two-Pointers ³¹ , Sliding Window ³ , Monotonic Stack ⁴⁷ , Disjoint Set ⁴⁹)
- LeetCode discuss (patterns lists) ⁵⁵
- Educative/Medium guides by Fahim ul Haq ² ³⁴
- Standard algorithm texts (implicit in complexities).

No single resource covers everything, so this report integrates multiple guides. If any detail was not found in sources, it is marked as standard knowledge. All key statements above are backed by the cited materials.

1 8 9 11 14 16 20 28 29 52 Data structures and algorithms study cheatsheets for coding interviews | Tech Interview Handbook

<https://www.techinterviewhandbook.org/algorithms/study-cheatsheet/>

2 12 15 17 18 19 22 24 34 35 36 37 38 39 40 42 43 44 45 46 48 50 53 54 Beyond 14 LeetCode Patterns: Advanced Patterns to Crack FAANG Interviews | by Fahim ul Haq | Apr, 2026 | Medium

<https://medium.com/@fahimulhaq/beyond-14-leetcode-patterns-advanced-patterns-to-crack-faang-interviews-4efcaaf5f15c>

3 Sliding Window Technique - GeeksforGeeks

<https://www.geeksforgeeks.org/dsa/window-sliding-technique/>

4 5 7 Array cheatsheet for coding interviews | Tech Interview Handbook

<https://www.techinterviewhandbook.org/algorithms/array/>

6 25 26 27 33 41 51 14 Patterns to Ace Any Coding Interview Question - Discuss - LeetCode

<https://leetcode.com/discuss/post/4039411/14-Patterns-to-Ace-Any-Coding-Interview-Question/>

10 31 Two Pointers Technique - GeeksforGeeks

<https://www.geeksforgeeks.org/dsa/two-pointers-technique/>

13 Monotonic Stack | Hello Interview

<https://www.hellointerview.com/learn/code/stack/monotonic-stack>

21 23 30 Graph cheatsheet for coding interviews | Tech Interview Handbook

<https://www.techinterviewhandbook.org/algorithms/graph/>

32 LeetCode 167 — Two Sum II — Input Array Is Sorted | by Eric Ness | Medium

<https://medium.com/@eric.christopher.ness/leetcode-167-two-sum-ii-input-array-is-sorted-c9b9b8ecea5>

47 How to Identify and Solve Monotonic Stack Problems ? - GeeksforGeeks

<https://www.geeksforgeeks.org/dsa/how-to-identify-and-solve-monotonic-stack-problems/>

49 Introduction to Disjoint Set (Union-Find Data Structure) - GeeksforGeeks

<https://www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/>

55 Important DSA Patterns | 100% Must-Know for Cracking Coding Interviews - Discuss - LeetCode

<https://leetcode.com/discuss/post/5908573/Important-DSA-Patterns-100-to-Crack-Coding-Interviews/>